Case Study 8: Password Policy Validation Engine

1. Problem Statement

Develop a Python program to validate a password according to a set of security rules. The program should check the password length and determine whether it contains uppercase letters, lowercase letters, digits, and special characters. It should display whether the password meets all the required rules.

2. Objectives

- To understand strings and string operations in Python.
- To use conditional statements for password validation.
- To check uppercase and lowercase letters.
- To identify digits and special characters.
- To use Boolean variables and logical operators.
- To display appropriate validation messages.

3. Assumptions

The following rules are used for this example implementation:

1. The password must contain at least 8 characters.
2. It must contain at least one uppercase letter.
3. It must contain at least one lowercase letter.

4. It must contain at least one digit.
5. It must contain at least one special character, such as `@`, `#`, or `!`.
6. The password is accepted only when all five rules are satisfied.

4. Algorithm

1. Start the program.
2. Ask the user to enter a sample password.
3. Check whether the password has at least 8 characters.
4. Check for uppercase letters, lowercase letters, digits, and special characters.
5. Check whether any character repeats three times consecutively.
6. Check whether the password contains prohibited substrings such as `password` or `1234`, ignoring letter case.
7. Display each failed rule.
8. If all rules are satisfied, display “Password accepted”; otherwise, display “Password rejected.”
9. Stop the program.

5. Flowchart

Start
Enter sample password
Check length and character categories
Check repetition and prohibited substrings
Are all rules satisfied?

Yes

Password accepted

No

Display failed rules

Stop

6. Pseudocode

```
None
START

INPUT password
```

```
CREATE empty list called errors

IF length of password < 8:
    ADD length error

CHECK for uppercase letters
CHECK for lowercase letters
CHECK for digits
CHECK for special characters

IF any character repeats 3 times consecutively:
    ADD repetition error

IF password contains "password" or "1234"
(case-insensitive):
    ADD prohibited-substring error

IF errors list is empty:
    DISPLAY "Password accepted"
ELSE:
    DISPLAY "Password rejected"
    DISPLAY each error

STOP
```

7. Python Source Code

Save the program as `password_validation.py`.

None

```
print("===== PASSWORD POLICY VALIDATION ENGINE =====")

password = input("Enter a sample password: ")
```

```
errors = []

# Rule 1: Minimum length
if len(password) < 8:
    errors.append("Password must contain at least 8 characters.")

# Rule 2: Uppercase letter
has_upper = False
for ch in password:
    if ch.isupper():
        has_upper = True

if not has_upper:
    errors.append("Include at least one uppercase letter.")

# Rule 3: Lowercase letter
has_lower = False
for ch in password:
    if ch.islower():
        has_lower = True

if not has_lower:
    errors.append("Include at least one lowercase letter.")

# Rule 4: Digit
has_digit = False
for ch in password:
    if ch.isdigit():
        has_digit = True

if not has_digit:
    errors.append("Include at least one digit.")

# Rule 5: Special character
has_special = False
for ch in password:
```

```
    if not ch.isalnum() and not ch.isspace():
        has_special = True

if not has_special:
    errors.append("Include at least one special character.")

# Rule 6: Three identical consecutive characters
has_repetition = False

for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        has_repetition = True

if has_repetition:
    errors.append(
        "Do not repeat the same character 3 times consecutively."
    )

# Rule 7: Prohibited substrings
lower_password = password.lower()

if "password" in lower_password or "1234" in lower_password:
    errors.append(
        "Password contains a prohibited substring."
    )

# Display results
print("\n===== VALIDATION REPORT =====")

if len(errors) == 0:
    print("Password accepted.")
    print("All policy rules are satisfied.")
else:
    print("Password rejected.")
    print("Failed rules:")
```

```
    for error in errors:
        print("-", error)

print("Validation completed.")
```

8. Test Cases

Test input	Expected result
`Hello@123`	Accepted
`hello123`	Rejected: uppercase and special character missing
`HELLO@123`	Rejected: lowercase letter missing
`Hi@1`	Rejected: minimum length not met
`Abc@aaa123`	Rejected: three consecutive `a` characters
`Password@123`	Rejected: contains prohibited substring `password`

`Abc@1234X`	Rejected: contains prohibited substring `1234`

Sample output 1: Valid password

```
None
===== PASSWORD POLICY VALIDATION ENGINE =====
Enter a sample password: Hello@123

===== VALIDATION REPORT =====
Password accepted.
All policy rules are satisfied.
Validation completed.
```

Sample output 2: Invalid password

```
None
===== PASSWORD POLICY VALIDATION ENGINE =====
Enter a sample password: hello123

===== VALIDATION REPORT =====
Password rejected.
Failed rules:
- Include at least one uppercase letter.
- Include at least one special character.
Validation completed.
```

9. Output Screenshot

```
print("===== PASSWORD POLICY VALIDATION ENGINE =====")

password = input("Enter a sample password: ")

errors = []

# Rule 1: Minimum length
if len(password) < 8:
    errors.append("Password must contain at least 8 characters.")

# Rule 2: Uppercase letter
has_upper = False
for ch in password:
    if ch.isupper():
        has_upper = True

if not has_upper:
    errors.append("Include at least one uppercase letter.")

# Rule 3: Lowercase letter
has_lower = False
for ch in password:
    if ch.islower():
        has_lower = True

if not has_lower:
    errors.append("Include at least one lowercase letter.")

# Rule 4: Digit
has_digit = False
for ch in password:
    if ch.isdigit():
        has_digit = True

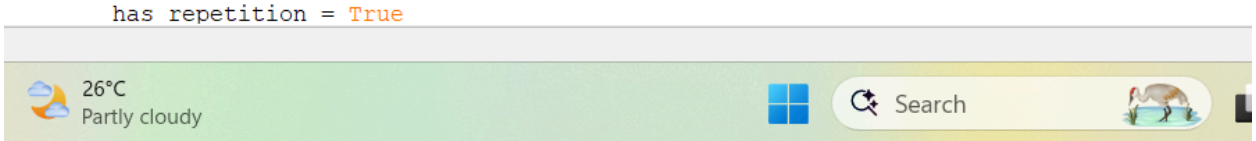
if not has_digit:
    errors.append("Include at least one digit.")

# Rule 5: Special character
has_special = False
for ch in password:
    if not ch.isalnum() and not ch.isspace():
        has_special = True

if not has_special:
    errors.append("Include at least one special character.")

# Rule 6: Three identical consecutive characters
has_repetition = False

for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        has_repetition = True
```



```
# Display results
print("\n===== VALIDATION REPORT =====")

if len(errors) == 0:
    print("Password accepted.")
    print("All policy rules are satisfied.")
else:
    print("Password rejected.")
    print("Failed rules:")

    for error in errors:
        print("-", error)

print("Validation completed.")
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>
======== RESTART: C:/Users/YOGITHA/OneDrive/Desktop/8.password policy.py =======
===== PASSWORD POLICY VALIDATION ENGINE =====
Enter a sample password: Hello@123

===== VALIDATION REPORT =====
Password accepted.
All policy rules are satisfied.
Validation completed.
>
======== RESTART: C:/Users/YOGITHA/OneDrive/Desktop/8.password policy.py =======
===== PASSWORD POLICY VALIDATION ENGINE =====
Enter a sample password: hello123

===== VALIDATION REPORT =====
Password rejected.
Failed rules:
- Include at least one uppercase letter.
- Include at least one special character.
Validation completed.
>
```

10. Conclusion

The Password Policy Validation Engine demonstrates how Python strings, loops, conditional statements, and lists can be used to validate sample passwords. It checks character categories,

length, repeated characters, and prohibited substrings, and explains why a sample password fails the specified rules.